

INDEX

I.	Introduction.....	1
II.	1st step: Catalogue or parent class creation.....	2
III.	2nd step: Electronic class creation	3
IV.	3rd step: BookProduct class creation	4
V.	4th step: Shopping class creation	4
VI.	5th step: Payment card class creation	4
VII.	6th step: Delivery service class creation	4
VIII.	7th step: Delivery service sample creation.....	4
IX.	8th step: Menu displayed creation I PART.....	4
X.	8th step: Menu displayed creation II PART.....	4
XI.	Conclusion.....	5
XII.	Other sources.....	6

Introduction

Even python first appearance was in 1991 has remained continue as the second most used programming language nowadays, as professor always mention on previous lectures, is a friendly programming although at the beginning is not for those who do not have prior computation skills, is the best program to start.

The importance of mastering the use of Python specifically as a programming tool, is indisputable, since python is widely used in the financial and economic industry to quickly perform complex computation; stock markets generate huge amounts of data, and Python in most of the time is used to import data on stock prices and generate strategies through algorithms to identify trading opportunities precisely in finance field. Additionally, in economics, the language can also be used for portfolio optimization, risk management, financial modeling, and visualization, welfare or any other social analysis and even fraud detention.

Throughout the report, a detailed explanation of the development of the shopping system is given based on the following recap guidelines: (i)The store has a catalogue, which has two product categories: Electronics & books.(ii) Books attributes: title, author, publisher, price, ISBN and language. (iii) Electronics attributes: brand, type, a display name, price, number of inventory, inventory status and 2 years warranty. (iv) The electronics category has subclasses: computers, tables and cellphones. And books have subclasses: computer science, mathematics and economics. (v) computer products are: laptops, macbook and chromebook. Tables products are: Amazon fire, ipad and samsung galaxy. Mobile phones are: iphone, samsung galaxy and google pixel and 5G devices (vi)computers have additional attributes, Bluetooth, WIFI, HDMI output (yes or no values). (vi) create a shopping cart class, which can be displayed anytime. (vii) delivery service with free shipping after \$100 or greater. (viii) display menu for customer choice. (viii) payment card class attributes: card number, expiry date, holder name, card balance (use `input()` function).

1st step: Catalogue or parent class creation

After comprehensive or getting the initial intuition of how the shopping system will be created, (i) we created the class catalogue which is the parent¹ or superclass. (ii) We defined the main attributes: electronics and books, which later will become the classes of the shopping system, and we made use of the initializer or constructor².

2nd step: Electronic class creation

¹ Superclass: is the parent of a class, it is the class from which the next classes created will inherit. If you don't specify a superclass explicitly, python automatically adds the class (object) as a parent class, which in this case is specified.

² The initializer: also known as constructor for the class. It indicates how data is stored internally and this (`__init__`) method is declared with a class and used to initialize the attribute of an object as soon as the object is formed.

Following the creation of the ElectronicProduct class, making use of the constructor class again, and we specified the instance variables or attributes³ (`self.brand`, `self.type`, `self.display_name`, `self.price`, `self.n_inventory_status`, `self.warranty`) with their respective parameters. We decided to create an electronics class first, since it will contain some attributes that will be inherited during the creation of the book class.

We made use of the string method to print ElectronicProduct data. Right after we added two functions definition⁴ since they will be called later for the shopping cart (`def buy_product(self)`), and the second function definition (`restock_inventory(self)`) will be called for the use of the object in the attribute of (`inventory_status`), we use some conditional operations in the first one, to ensure that if the number of inventory is greater than 0, then the number of inventory will be reduced by 1. We made use of the 'if statement' to condition if the inventory is less than 15 then it prints or shows "Low stock" and on the else it will print "product out of the stock". The same procedure is followed in the second function definition (`restock_inventory(self)`), the 'if statement' will be used to add 10 items after the consumer has bought something but only if the inventory is less than 5 items.

*those conditional definition functions are placed within the ElectronicProduct class, since it only applies to electronic products, even though both are instance variables⁵ since they are tied to an instance when the object or delivery service sample will be defined later in step 7.

Lastly, we created the remaining subclasses `tables(class tables(ElectronicProducts))` and `cellphone(class Tables (ElectronicProduct))`, which you can clearly see on appendix section, that were formed with inheritance⁶, inheriting all the attributes of the parent class. With expectation of the subclass (`class Computer (ElectronicProduct)`) which added the attributes remaining (`self.bluetooth`, `self.wifi`, `self.hmdi_output`, `self.ram_size`)

3rd step: BookProduct class creation

We created a new complete class with the catalog parent, in which we assign new complete attributes (`self.area_of_study`, `self.title`, `self.publisher`, `self.author`, `self.price`, `self.isbn`, `self.language`). In this class, we made use of polymorphism⁷, even though the attributes (`self.n_inventory`) and (`self.inventory_status`) are not defined in the constructure, we added two function definitions to specify the behavior of the number of inventory and inventory status. It was essential to specify them apart from the ElectronicProduct class since the conditions are different.

4th step: Shopping Cart class creation

We created the shopping Carts class, in which we specified a default attribute⁸

³ Attribute: the data held by an object is represented by its attributes or instance variables.

⁴ Function definition: is a block of code which only runs when it is called (`def(self)`)

⁵ Instance variables: are used to define the behaviour of the instance or object meanwhile the class variable defines the behaviour of the class, synonym of attribute

⁶ Inheritance: allows one class to inherit data or behaviour from another class.

⁷ Polymorphism: is the ability to run a method with different objects, each of them appear to perform the same function, however the way the method is implemented depends on the object's class.

⁸ Default attribute: there are two ways to create a default attribute, you can define it at the beginning in the initializer (`__init__`) or in the attributes.

(`self.item=[ ]`), in this case the default attribute is defined in the same attribute as parameter and not within the constructor. The only problem defining the default attribute out as a parameter of the default rather than the constructor is that we won't be able to change our default item variable during the object creation, that is why it is an empty value or not specified value. We then created the function definition (`def add_to_cart(self, product)`) and (`display_cart(self)`), in the 'add to cart' function we used the append method⁹, joint with the attribute items and the creation of the parameter product since it will be called later in the menu displayed creation. Lastly, we defined the function (`def clear_cart(self,)`) adding a default variable in a way to show a non defined value as parameter, since it will clear or erase everything we have in our cart shopping right after the transaction is completed.

5th step: Payment card class creation

In this step, we created a basic PaymentClass card, with their respective attributes (`self.card_number`, `self.expiry_date`, `self.card_holder_name`, `self.card_balance`) since they were not already defined previously (so can not use inheritance), their respective parameters are the same.

In this step, we made use of encapsulation¹⁰ in the attributes, (`self.__card_number`, `self.__expiry_date`, `self.__card_holder_name`) to make it private since is sensitive customer information, with the exception of the card balance attribute (`self.card_balance`) since we want it to be public, because it is used in the menu displayed part when consumer will specify their amount of money they have in their cards. We defined the getter and setters, and added double underscores, decorators weren't added for obvious reasons.

It was created using a definition function by using the conditional statement in a way that if the total order is greater than the card balance, then the customer payment is declined, since customer doesn't have enough money to pay; otherwise the total order is a true value if the customer card balance has more money, we made use of boolean values for this part¹¹.

6th step: Delivery service class creation

Along this class creation, we created the definition function (`def calculate_shipping_costt(self,total_order)` and we used conditional statements and some operators to assign their respective value, to see that if the total order is greater than 100, the shipping will be free.

Within this step we created the class address, we define the attributes (`self.street`, `self.city`, `self.state`,`self.zip_code`), this is because these attributes will be called in the menu displayed, when the customer chooses option 3, as you see in the creation of this class we do not make use of inheritances (since there nothing to inherit from previous classes).

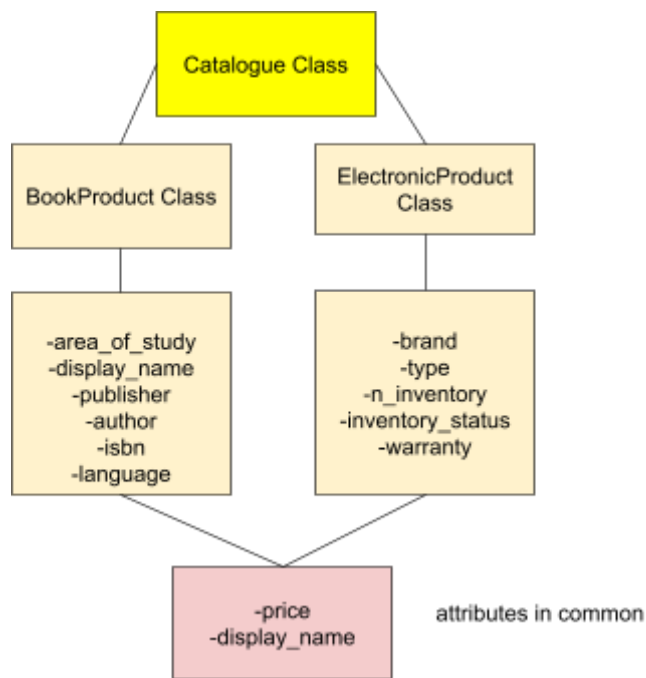
7th step: Delivery service sample creation

The object creation was not so complex, we follow the attributes which previously were defined on the Electronic class and BookProduct class, as you can observe the attributes are slightly different in both classes. We defined random prices for each product, random inventory number and random inventory status, etc.

⁹ Append: (`append()`) is a function used to add an item to the end of the list, the append method is a member of the list of the object and is used to modify the contents of an existing list by adding a new element.

¹⁰ Encapsulation : is applied to hide the class structure from external users.

¹¹ Boolean values: can be true or false.



8th step: Menu displayed creation I PART

The first part of the menu displayed only contains the option 1 “view products” which is going to display another menu for electronics and book products, and then its different sub products to finally display their characteristics. As you can see this part is mostly conform by printing all the instructions that the consumer must follow in order to buy their preferred product.

8th step: Menu displayed creation II PART

The second part of the menu displayed contains option 2 “buy a product”, option 3 “view shopping cart and checkout” and last option “exit”.

In this part of the project we faced a huge problem that we couldn’t fix at the begging:

Choosing option 2 without having previously checked option 1 doesn’t affect at all the chronology of menu displayed, the costumer can still buy a product without previously seen the products information (option 1), but if the customer decide from the onset of the purchasing procedure to choose option 3 “view option cart and checkout “ without previously entering to option 2 “buy a product” Python throws you *NameError*, making mention that you have to first define ‘cart’. Defining that cart is equal to shopping cart class in the beginning of the menu display, (`cart = shoppingCart()`), we ended up solving that issue.

Conclusion

The first challenge we faced was to get the structure intuition of how it will be conformed. Even though the beginning of the shopping system was clarified in the guidelines provided in this sense the creations of classes was the easiest part, later it started become complex in the way where to defined or create the definition functions specifically, ‘buy product’ (`def buy_product(self)`) and ‘restock inventory’ definitions functions, (`def restock_inventory(self)`)

since it was planned to be called later within the menu display. We struggled figuring out where definition functions were necessary to be located, as class variable or instance variable whether they had to be within a respective class and linked to the delivery service sample.

Another aspect that we had to take care the whole time is the spelling, the constant checking that our variables were all set up, running our project constantly to check if we were good with the structure; sometimes we used to forget the correlations or how the sample variables and classes variable were they related, so it was necessary to “debug” one by one to verify that those variable really came from where we need it. At the end, when we were almost done with the project, we got 6 errors approximate, we had to check it again and it was a space issue, nothing serious. Finally, when we were done we forgot to add some encapsulation to getters and setters functions to convert them in private, so we had to modify it being careful what functions were necessary to privatize.

Other sources

Python (2023). Learn python programming :Object oriented programming (online). United Kingdom; Consulted on May 5th, 2024. Available in: <https://pythonbasics.org/>

python(2023). Python tutorial:multiple inheritance & private variables (online). United Kingdom; Consulted on April 11st, 2024. Available in: <https://docs.python.org/3/tutorial/classes.html#multiple-inheritance>

Rizel Scarlett(2023). Why Python keeps growing, explained (online). Stanford University; Consulted on May 7th, 2024. Available in: <https://github.blog/2023-03-02-why-python-keeps-growing-explained/>

w3resource(2023). Python Object-OrientedProgramming:shopping cart class with item management and total calculation (online). Consulted on May 5tht, 2024. Available in: <https://www.w3resource.com/python-exercises/oop/python-oop-exercise-8.php>

github(2023). 30 days python(online). Consulted on May 7th, 2024. Available in: <https://github.com/Asabeneh/30-Days-Of-Python>

